

API REFERENCE

Internship Progress Dashboard

Version 1.0 — April 2026

Field	Value
Base URL (Development)	http://localhost:8000/api
Interactive Docs (Swagger)	http://localhost:8000/api/docs
Authentication	Bearer JWT (Authorization: Bearer <token>)
Token Algorithm	HS256
Token Expiry	1440 minutes (24 hours)
Data Format	JSON (application/json)
Framework	FastAPI 0.104.1 + Python 3.11+
Database	MongoDB 6+ via Beanie ODM

HTTP Method Color Key: GET POST PUT DELETE

1. Overview

1.1 Authentication

All endpoints except `/auth/register` and `/auth/login` require a valid JWT token. Include it in the request header:

```
|Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Obtain the token from `POST /auth/login` or `POST /auth/register`. The token encodes the user ID and role (student or mentor). It expires after 1440 minutes (24 hours). There is no refresh endpoint — re-login is required on expiry.

1.2 Roles

Role	Value in Token	Description
Mentor	mentor	Creates tasks, manages students, reviews reports. Acts as system admin — no separate admin role.
Student	student	Updates own task status, submits weekly reports, views own analytics.

1.3 Standard Error Responses

HTTP Code	When It Occurs	Example Body
400	Validation error / duplicate resource	<code>{"detail": "Email already registered"}</code>
401	Missing, invalid, or expired JWT	<code>{"detail": "Could not validate credentials"}</code>
403	Authenticated but insufficient role/ownership	<code>{"detail": "Only mentors can create tasks"}</code>
404	Resource not found	<code>{"detail": "Task not found"}</code>
422	Request body schema violation (Pydantic)	<code>{"detail": [{"loc": ["body", "email"], "msg": "..."}]}</code>
500	Unhandled server / database error	<code>{"detail": "Internal server error"}</code>

1.4 Date & Time

- All timestamps are stored and returned in UTC.
- Format: ISO 8601 — e.g. 2026-04-26T14:01:00.000000
- Frontend is responsible for converting to the user's local timezone.

2. Authentication [/auth]

POST **/auth/register** — Register a new user

Access: Public (no token required)

Request Body

Field	Type	Required	Description
name	string	Yes	Full name of the user
email	string	Yes	Valid email address. Must be unique in the system.
password	string	Yes	Password (max 72 chars — bcrypt limit). Stored hashed.
role	string	No	Either 'student' or 'mentor'. Default: 'student'.

Request Example

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "password": "SecurePass123",
  "role": "student"
}
```

Response — 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0...",
  "token_type": "bearer",
  "user": {
    "id": "6630a1b2c3d4e5f6a7b8c9d0",
    "name": "Rahul Sharma",
    "email": "rahul@example.com",
    "role": "student",
    "avatar": null,
    "bio": null,
    "skills": [],
    "mentor_id": null,
    "created_at": "2026-04-26T14:00:00.000000"
  }
}
```

Error Cases

Code	Condition
400	Email already registered

POST **/auth/login** — Login and receive JWT token

Access: Public (no token required)

Request Body

Field	Type	Required	Description
email	string	Yes	Registered email address
password	string	Yes	Account password

Request Example

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "rahul@example.com",
  "password": "SecurePass123"
}
```

Response — 200 OK

Same structure as /register response (access_token + token_type + user object).

Error Cases

Code	Condition
401	Email not found or password incorrect

GET /auth/me — Get current authenticated user profile

Access: Any authenticated user

Response — 200 OK

```
{
  "id": "6630a1b2c3d4e5f6a7b8c9d0",
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "role": "student",
  "avatar": null,
  "bio": "Full-stack intern at Acme",
  "skills": ["Python", "React"],
  "mentor_id": "6630a1b2000000000000000001",
  "created_at": "2026-04-26T14:00:00.000000"
}
```

Error Cases

Code	Condition
401	Token missing, invalid or expired

3. Tasks [/tasks]

Tasks are created by mentors and assigned to students. Mentors have full control; students can only update the status of tasks assigned to them.

GET **/tasks/** — List tasks (role-filtered)

Access: Any authenticated user

Query Parameters

Parameter	Type	Required	Description
status	string	No	Filter by status: todo in_progress completed
search	string	No	Keyword search across title, description, and tags

Behavior by Role

Role	Returns
Mentor	All tasks created by this mentor (regardless of which student they're assigned to)
Student	All tasks assigned to this student

Response — 200 OK (Array)

```
[
  {
    "id": "6630b1000000000000000001",
    "title": "Build REST API",
    "description": "Implement all CRUD endpoints with FastAPI",
    "status": "in_progress",
    "priority": "high",
    "assigned_to": "6630a1b2c3d4e5f6a7b8c9d0",
    "created_by": "6630a1b20000000000000001",
    "due_date": "2026-05-01T00:00:00.000000",
    "tags": ["backend", "fastapi"],
    "created_at": "2026-04-20T09:00:00.000000",
    "updated_at": "2026-04-22T11:00:00.000000"
  }
]
```

POST **/tasks/** — Create a new task

Access: Mentor only

Request Body

Field	Type	Required	Description
title	string	Yes	Task title
description	string	No	Detailed description of the task

Field	Type	Required	Description
status	string	No	todo in_progress completed. Default: todo
priority	string	No	low medium high. Default: medium
assigned_to	string	Yes	MongoDB ObjectId of the student to assign
due_date	string	No	ISO 8601 datetime string. Optional deadline.
tags	array	No	List of string tags. Default: []

Side Effects

- Creates an in-app notification for the assigned student with type: task_assigned

Request Example

```
POST /api/tasks/
Authorization: Bearer <mentor-token>
Content-Type: application/json

{
  "title": "Build REST API",
  "description": "Implement all CRUD endpoints with FastAPI",
  "priority": "high",
  "assigned_to": "6630a1b2c3d4e5f6a7b8c9d0",
  "due_date": "2026-05-01T00:00:00",
  "tags": ["backend", "fastapi"]
}
```

Response — 200 OK

Returns the created task object (same shape as list item above).

Error Cases

Code	Condition
403	Authenticated user is a student
401	No token or invalid token

PUT `/tasks/{task_id}` — Update a task

Access: Both roles (different permissions)

Path Parameters

Parameter	Type	Description
task_id	string	MongoDB ObjectId of the task

Request Body

Field	Type	Mentor	Student	Description
title	string	Yes	Ignored	Updated task title
description	string	Yes	Ignored	Updated description

Field	Type	Mentor	Student	Description
status	string	Yes	Yes (only field)	todo in_progress completed
priority	string	Yes	Ignored	low medium high
due_date	string	Yes	Ignored	ISO 8601 datetime string
tags	array	Yes	Ignored	Replacement tag list

Note: Students sending fields other than 'status' will receive a 200 response, but those fields are silently ignored. Only 'status' is applied.

Error Cases

Code	Condition
403	Student trying to update another student's task / Mentor accessing another mentor's task
404	task_id not found or invalid ObjectId format

DELETE `/tasks/{task_id}` — Delete a task

Access: Mentor only (creator)

Path Parameters

Parameter	Type	Description
task_id	string	MongoDB ObjectId of the task to delete

Response — 200 OK

```
{ "message": "Task deleted successfully" }
```

Error Cases

Code	Condition
403	Authenticated user is a student, or mentor is not the task creator
404	task_id not found

4. Reports [/reports]

Students submit weekly Markdown reports. Mentors and students can both add comments. Mentors can only read reports from their assigned students.

GET **/reports/** — List reports (role-filtered)

Access: Any authenticated user

Behavior by Role

Role	Returns
Mentor	Reports submitted by students assigned to this mentor
Student	Only this student's own reports

Response — 200 OK (Array)

```
[
  {
    "id": "6630c2000000000000000001",
    "title": "Week 3 Progress",
    "content": "## Summary\nCompleted the API module...",
    "student_id": "6630a1b2c3d4e5f6a7b8c9d0",
    "week_number": 3,
    "comments": [
      {
        "id": "uuid-string",
        "user_id": "6630a1b20000000000000001",
        "user_name": "Prof. Mehta",
        "content": "Great progress! Keep it up.",
        "created_at": "2026-04-22T10:00:00.000000"
      }
    ],
    "created_at": "2026-04-21T08:00:00.000000",
    "updated_at": "2026-04-22T10:00:00.000000"
  }
]
```

GET **/reports/{report_id}** — Get a single report

Access: Mentor (assigned students) | Student (own reports only)

Path Parameters

Parameter	Type	Description
report_id	string	MongoDB ObjectId of the report

Error Cases

Code	Condition
403	Student tries to access another student's report
404	report_id not found

POST `/reports/` — Submit a weekly report

Access: Student only

Request Body

Field	Type	Required	Description
title	string	Yes	Report title (e.g. 'Week 3 Progress')
content	string	Yes	Report body. Markdown supported.
week_number	integer	Yes	Internship week number (manually entered)

Side Effects

- If student has a mentor_id set, creates a notification for that mentor with type: report_submitted

Error Cases

Code	Condition
403	Authenticated user is a mentor

PUT `/reports/{report_id}` — Update own report

Access: Student (owner only)

Request Body

Same fields as POST `/reports/` — all three fields (title, content, week_number) must be supplied.

Error Cases

Code	Condition
403	Student is not the report owner
404	report_id not found

DELETE `/reports/{report_id}` — Delete own report

Access: Student (owner only)

Response — 200 OK

```
{ "message": "Report deleted" }
```

Error Cases

Code	Condition
403	Student is not the report owner
404	report_id not found

POST `/reports/{report_id}/comments` — Add a comment to a report

Access: Any authenticated user

Path Parameters

Parameter	Type	Description
report_id	string	MongoDB ObjectId of the report

Request Body

Field	Type	Required	Description
content	string	Yes	Comment text

Side Effects

- If the commenter is a mentor and the report owner is a different user, creates a notification for the student with type: feedback_received

Response — 200 OK

Returns the full updated report object including the new comment in the comments array.

DELETE `/reports/{report_id}/comments/{comment_id}` — Delete own comment

Access: Any authenticated user (own comment only)

Path Parameters

Parameter	Type	Description
report_id	string	MongoDB ObjectId of the report
comment_id	string	UUID string of the comment (from comment.id)

Response — 200 OK

Returns the full updated report object with the comment removed.

Error Cases

Code	Condition
404	report_id not found, comment_id not found, or comment does not belong to current user

5. Analytics [/analytics]

GET **/analytics/** — Get role-specific dashboard analytics

Access: Any authenticated user

Returns different response shapes depending on the user's role. The role field in the response indicates which schema applies.

Mentor Response Schema

```
{
  "role": "mentor",
  "total_students": 3,
  "total_tasks": 12,
  "task_status": {
    "completed": 5,
    "in_progress": 4,
    "todo": 3
  },
  "completion_rate": 41.7,
  "weekly_activity": [
    { "week": "W1", "tasks_created": 3, "tasks_completed": 1 },
    { "week": "W2", "tasks_created": 2, "tasks_completed": 2 },
    ... // 7 weeks total
  ],
  "student_progress": [
    { "name": "Rahul Sharma", "total": 4, "completed": 2, "rate": 50.0 },
    { "name": "Priya Patel", "total": 8, "completed": 3, "rate": 37.5 }
  ]
}
```

Student Response Schema

```
{
  "role": "student",
  "total_tasks": 8,
  "total_reports": 4,
  "task_status": {
    "completed": 3,
    "in_progress": 2,
    "todo": 3
  },
  "completion_rate": 37.5,
  "weekly_activity": [
    { "week": "W1", "tasks_completed": 1, "reports_submitted": 1 },
    ... // 7 weeks total
  ],
  "priority_breakdown": { "high": 3, "medium": 4, "low": 1 },
  "feedback_received": 6
}
```

Field	Scope	Description
weekly_activity	Both	7 weeks ending at current UTC time, W1 = oldest, W7 = current
completion_rate	Both	Rounded to 1 decimal. Returns 0.0 if no tasks exist.

Field	Scope	Description
feedback_received	Student	Total number of comments received across all student's reports
student_progress	Mentor	One entry per assigned student with individual completion rate

6. Users [/users]

GET **/users/students** — List mentor's assigned students

Access: Mentor only

Response — 200 OK (Array)

```
[
  {
    "id": "6630a1b2c3d4e5f6a7b8c9d0",
    "name": "Rahul Sharma",
    "email": "rahul@example.com",
    "skills": ["Python", "React"],
    "bio": "Full-stack intern"
  }
]
```

Error Cases

Code	Condition
403	Authenticated user is a student

GET **/users/all-students** — List all students (for assign/unassign UI)

Access: Mentor only

Response — 200 OK (Array)

```
[
  {
    "id": "6630a1b2c3d4e5f6a7b8c9d0",
    "name": "Rahul Sharma",
    "email": "rahul@example.com",
    "mentor_id": "6630a1b2000000000000000001"
  },
  {
    "id": "6630a1b2c3d4e5f6a7b8c9d1",
    "name": "Priya Patel",
    "email": "priya@example.com",
    "mentor_id": null
  }
]
```

PUT **/users/assign-toggle/{student_id}** — Toggle assign / unassign a student

Access: Mentor only

Path Parameters

Parameter	Type	Description
student_id	string	MongoDB ObjectId of the student to assign or unassign

Toggle Logic

Current State	Action	Result
student.mentor_id is null	Assign	Sets student.mentor_id = current mentor's ID
student.mentor_id is set	Unassign	Sets student.mentor_id = null

Warning: If a student is already assigned to another mentor, the toggle will overwrite that assignment (no conflict check). Handle this in the frontend.

Response — 200 OK

```
{ "message": "Student Assigned" }
// or
{ "message": "Student Unassigned" }
```

Error Cases

Code	Condition
403	Authenticated user is a student
404	student_id not found or invalid ObjectId

PUT `/users/profile` — Update own profile

Access: Any authenticated user

Request Body — All fields optional (partial update)

Field	Type	Description
name	string	Updated display name
bio	string	Short biography / introduction
skills	array	List of skill strings, e.g. ['Python', 'React']
avatar	string	URL string pointing to profile image

Response — 200 OK

```
{
  "id": "6630alb2c3d4e5f6a7b8c9d0",
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "role": "student",
  "bio": "Full-stack intern at Acme",
  "skills": ["Python", "React", "MongoDB"],
  "avatar": "https://cdn.example.com/avatar.jpg",
  "created_at": "2026-04-26T14:00:00.000000"
}
```

7. Notifications [/notifications]

Notifications are automatically created by the system on specific events. Users cannot manually create notifications.

Notification Types

Type Value	Trigger Event	Recipient
task_assigned	Mentor creates a task assigned to a student	Student
report_submitted	Student submits a weekly report	Mentor (if assigned)
feedback_received	Mentor comments on a student's report	Student

GET `/notifications/` — Get recent notifications

Access: Any authenticated user

Returns the latest 30 notifications for the current user, sorted by creation date descending.

Response — 200 OK (Array)

```
[
  {
    "id": "6630d3000000000000000001",
    "title": "New Task Assigned",
    "message": "You have been assigned a new task: \"Build REST API\"",
    "type": "task_assigned",
    "read": false,
    "created_at": "2026-04-26T14:05:00.000000"
  }
]
```

GET `/notifications/unread-count` — Get unread notification count

Access: Any authenticated user

Response — 200 OK

```
{ "count": 5 }
```

PUT `/notifications/{notif_id}/read` — Mark a notification as read

Access: Any authenticated user (own notifications only)

Path Parameters

Parameter	Type	Description
notif_id	string	MongoDB ObjectId of the notification

Response — 200 OK

```
{ "message": "Marked as read" }
```

Note: If the notification doesn't belong to the current user, the request is silently ignored (no error returned).

PUT **/notifications/read-all** — Mark all notifications as read

Access: Any authenticated user

Response — 200 OK

```
{ "message": "All notifications marked as read" }
```

Note: Only marks notifications belonging to the current user. Other users' notifications are not affected.

8. Quick Reference — All Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, get JWT token
GET	/api/auth/me	Any Auth	Get own profile
GET	/api/tasks/	Any Auth	List tasks (role-filtered). Params: status, search
POST	/api/tasks/	Mentor	Create task + notify student
PUT	/api/tasks/{id}	Both	Mentor: full update; Student: status only
DELETE	/api/tasks/{id}	Mentor	Delete own task
GET	/api/reports/	Any Auth	List reports (role-filtered)
GET	/api/reports/{id}	Any Auth	Get single report
POST	/api/reports/	Student	Submit weekly report + notify mentor
PUT	/api/reports/{id}	Student	Update own report (all fields required)
DELETE	/api/reports/{id}	Student	Delete own report
POST	/api/reports/{id}/comments	Any Auth	Add comment + notify student if mentor
DELETE	/api/reports/{id}/comments/{cid}	Any Auth	Delete own comment
GET	/api/analytics/	Any Auth	Role-specific dashboard data
GET	/api/users/students	Mentor	Own assigned students
GET	/api/users/all-students	Mentor	All students (for assignment UI)
PUT	/api/users/assign-toggle/{id}	Mentor	Toggle assign/unassign student
PUT	/api/users/profile	Any Auth	Update own profile (partial)
GET	/api/notifications/	Any Auth	Last 30 notifications, newest first
GET	/api/notifications/unread-count	Any Auth	{ count: N }
PUT	/api/notifications/{id}/read	Any Auth	Mark single notification read
PUT	/api/notifications/read-all	Any Auth	Mark all notifications read

9. Revision History

Version	Date	Author	Changes
1.0	April 2026	Development Team	Initial API Reference — all modules documented

--- End of Document ---